

Docker Container High CPU Usage SOP

Objective

Investigate and reduce excessive CPU usage in Docker containers while maintaining application stability and service performance.

Scope

Applicable to Docker-based services, containerized application deployments, DevOps teams, SRE teams, platform teams, and synthetic incident workflows across development, staging, and production-like environments.

Pre-requisites

1. Access to Docker host, Docker CLI, container runtime logs, and the target container or image name.
2. Permission to inspect containers, images, volumes, networks, resource limits, and compose files where applicable.
3. Access to application logs, deployment artifacts, Dockerfile, environment variables, and registry credentials.
4. Monitoring access for host CPU, memory, disk, container restart count, and container health checks.
5. Escalation contact for application, DevOps, security, registry, infrastructure, and platform teams.

Incident Metadata

Field	Value
Ticket ID	T020
Issue	Docker container high CPU usage
Category	Performance
Service	Docker
Severity	Low
Status	Resolved
Description	Container consuming excessive CPU
Expected Resolution	Optimize application code and limit resources

Related Log Evidence

Log ID	Timestamp	Level	Message	Error Code
N/A	N/A	N/A	No direct log evidence found in logs CSV for this ticket.	N/A

Procedure

1. Identify the container, host, CPU usage pattern, start time, and impacted application endpoints.
2. Run docker stats to measure CPU, memory, network, and block IO usage.
3. Review application logs for traffic spikes, loops, stuck jobs, or repeated retries.
4. Inspect container resource limits and compare usage against expected workload.
5. Check recent deployment, configuration, or dependency changes.
6. Profile the application or collect thread dumps where supported.
7. Apply mitigation such as scaling, rate limiting, fixing code loops, or setting CPU limits.
8. Monitor CPU usage and application latency after the fix.

Common Commands

```
docker ps -a
docker logs <container-name-or-id> --tail=100
docker inspect <container-name-or-id>
docker stats --no-stream
docker images
docker network ls && docker network inspect <network-name>
docker volume ls && docker volume inspect <volume-name>
docker system df
```

Troubleshooting Matrix

Issue	Possible Cause	Resolution
High CPU usage	Application loop or inefficient code	Optimize code or stop runaway process
CPU spike during traffic	Increased load	Scale service or apply rate limits
Host CPU saturated	Too many containers on host	Move workloads or add capacity
Container affects host	No CPU limits	Set CPU quotas or compose limits

Best Practices

1. Set CPU limits for non-critical workloads.
2. Monitor container CPU trends and spikes.
3. Profile services before scaling blindly.
4. Alert on sustained high CPU with latency impact.

References

Docker Docs | Internal Tickets CSV | Internal Logs CSV | Container Runtime Logs | Monitoring Dashboards | DevOps Runbook